C++ COMPLETE REFERENCE LIBRARY • MODULE 14

14 FRIEND FUNCTIONS AND CLASSES — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 friend functions and rules
3. **Theory Chapter:** 02 bridging two classes and friend class
4. **Theory Chapter:** 03 encapsulation and friendship properties
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

14 — Friend Functions and Friend Classes

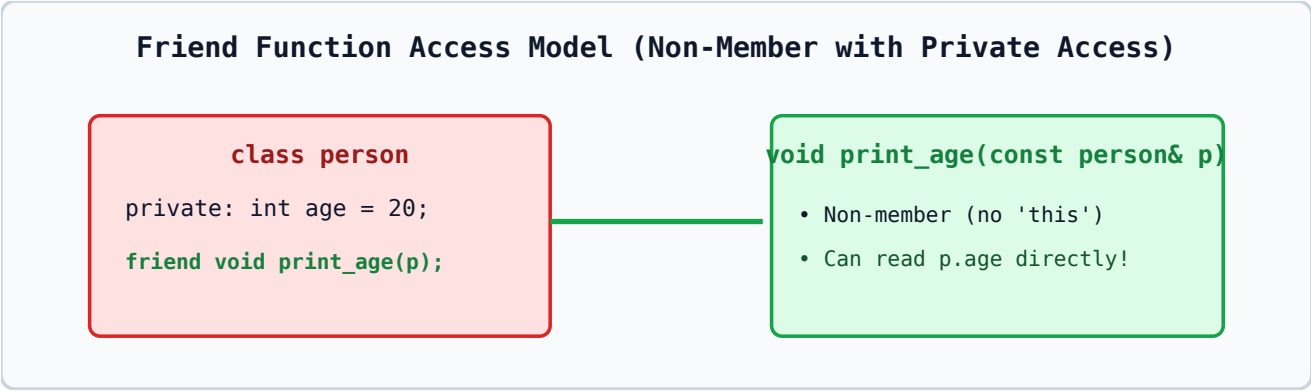
This module covers non-member `friend` functions, bridging private data across two independent classes, `friend class` declarations, and friendship non-transitivity/non-inheritance rules.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_friend_functions_and_rules.md` (./01_THEORY/01_friend_functions_and_rules.md)
  - `02_bridging_two_classes_and_friend_class.md` (./01_THEORY/02_bridging_two_classes_and_friend_class.md)
  - `03_encapsulation_and_friendship_properties.md` (./01_THEORY/03_encapsulation_and_friendship_properties.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_friend_function_single_class.cpp` (./02_PROGRAMS/01_friend_function_single_class.cpp) (Category A)

  - `02_friend_function_two_classes.cpp` (./02_PROGRAMS/02_friend_function_two_classes.cpp) (Category A)
  - `03_friend_class_demonstration.cpp` (./02_PROGRAMS/03_friend_class_demonstration.cpp) (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_friend_access_model.svg` (./05_DIAGRAMS/diagram_01_friend_access_model.svg)

01 — Friend Functions: Syntax and Core Rules



Friend Access Model

1. Definition

A `friend` **function** of a class is a **non-member function** that is granted special permission by the class to access its `private` and `protected` members through an object of that class.

2. Key Rules of `friend` Functions

1. **Not in Class Scope**: A `friend` function is **not** a member function of the class, so it is not called with `obj.func()` and has **no** `this` **pointer**.
2. **Access Specifier Independence**: Placing `friend void show(const person& p);` inside `private:`, `protected:`, or `public:` has the exact same effect.

02 — Bridging Two Classes and `friend class`

1. Forward Declaration When Bridging Two Classes

To write a single `friend` function `add_incomes(const teacher&, const researcher&)` that accesses `private` members of two different classes, forward-declare the second class (`class researcher;`) before defining the first class (`class teacher`).

2. `friend class`

Declaring `friend class B;` inside `class A` grants **every member function of** `B` access to `A` 's `private` and `protected` members.

03 — Four Mathematical Properties of Friendship in C++ (`[class.friend]`)

1. **Not Symmetric (Not Mutual)**: If `A` declares `B` as a friend, `B` can access `A`'s private members, but `A` **cannot** access `B`'s private members unless `B` also declares `A` as a friend.
2. **Not Transitive**: If `A` is a friend of `B`, and `B` is a friend of `C`, `A` is **not** automatically a friend of `C`.
3. **Not Inherited**: If `Base` is a friend of `X`, `Derived : public Base` is **not** a friend of `X`.
4. **Not Affected by `private:` / `public:` Labels**: Friendship declarations ignore enclosing access specifiers.

Visual Concept & Memory Diagrams (14 FRIEND FUNCTIONS AND CLASSES)

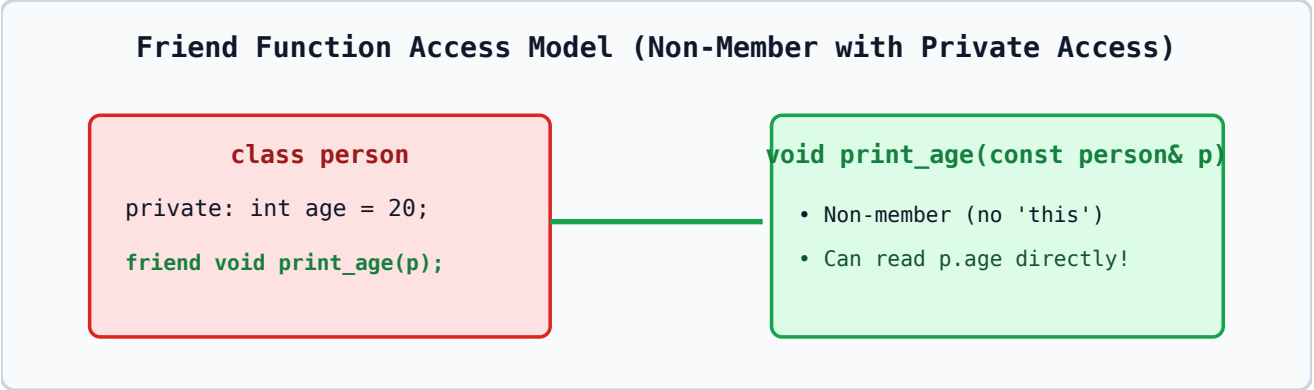


Diagram: diagram 01 friend access model

Annotated C++ Programs (14 FRIEND FUNCTIONS AND CLASSES)

Program: `01_friend_function_single_class.cpp`

C++ SOURCE FILE • 01_friend_function_single_class.cpp

```cpp
// Program Title: Friend Function Accessing Private Member of Classroom 'person' Class
// Topic: Friend Functions and Classes
// Purpose: Show how a non-member friend function reads private data of a person object.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_friend_functions_and_rules.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    void input(int a) {
        age = a;
    }

    friend void print_age(const person& p);
};

void print_age(const person& p) {
    cout << "Private age accessed via friend function: " << p.age << endl;
}

int main() {
    person p;
    p.input(20);
    print_age(p);
    return 0;
}

/*
Expected Deterministic Output:
Private age accessed via friend function: 20
*/
```

Program: `02_friend_function_two_classes.cpp`

C++ SOURCE FILE • 02_friend_function_two_classes.cpp

```cpp
// Program Title: Friend Function Bridging Two Classes Using Forward Declaration
// Topic: Friend Functions and Classes — Multi-Class Friend
// Purpose: Sum private values of two distinct classes (alpha and beta) using a common friend function.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_bridging_two_classes_and_friend_class.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class beta; // Forward declaration

class alpha {
private:
    int x;

public:
    alpha(int v) : x(v) {}
    friend int add_both(const alpha& a, const beta& b);
};

class beta {
private:
    int y;

public:
    beta(int v) : y(v) {}
    friend int add_both(const alpha& a, const beta& b);
};

int add_both(const alpha& a, const beta& b) {
    return a.x + b.y;
}

int main() {
    alpha a(35);
    beta b(65);
    cout << "Sum of private members: " << add_both(a, b) << endl;
    return 0;
}

/*
Expected Deterministic Output:
Sum of private members: 100
*/
```

Program: `03_friend_class_demonstration.cpp`

C++ SOURCE FILE • 03_friend_class_demonstration.cpp

```cpp
// Program Title: Friend Class Accessing Private Data of Another Class
// Topic: Friend Functions and Classes — friend class
// Purpose: Demonstrate declaring 'friend class result_printer;' inside 'class student'.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_bridging_two_classes_and_friend_class.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class student {
private:
    int roll;
    int marks;

public:
    student(int r, int m) : roll(r), marks(m) {}

    friend class result_printer;
};

class result_printer {
public:
    void show(const student& s) const {
        cout << "Roll: " << s.roll << ", Marks: " << s.marks << endl;
    }
};

int main() {
    student s(101, 94);
    result_printer rp;
    rp.show(s);
    return 0;
}

/*
Expected Deterministic Output:
Roll: 101, Marks: 94
*/
```

Conceptual Questions — Module 14: Friend Functions and Classes

Q1. Explain why C++ friendship is neither symmetric, nor transitive, nor inherited. **Q2.** Does a `friend` function violate encapsulation, or does it support controlled encapsulation?

Debugging Questions — Module 14

Q4. Why does writing `cout << age;` inside a `friend` function fail to compile?

Predict the Output Questions — Module 14

Q3. Does placing `friend void f(A&);` inside the `private:` section of `class A` prevent `main()` from calling `f(obj)`?

Programming Questions — Module 14

Q5. Write a C++23 classroom program using a `friend` function `swap_private(class_a&, class_b&)` to swap the private integers of two different classes.

Detailed Solutions — Conceptual Questions (Module 14)

- **Q1 Solution**: Friendship is an explicit per-class grant (`[class.friend]`). `A` trusting `B` does not mean `B` trusts `A` (non-symmetric), nor does `A` trust `B`'s friends (non-transitive) or `B`'s derived classes (non-inherited).
- **Q2 Solution**: Controlled friendship (such as `operator<<` or binary operators bridging two classes) avoids adding public getters/setters that would expose internal representation to the entire program.

Detailed Solutions — Predict the Output & Debugging (Module 14)

- **Q3 Solution**: **No!** A `friend` declaration is unaffected by whether it appears under `private:`, `protected:`, or `public:`. `main()` can call `f(obj)` normally.
- **Q4 Solution**: Because a `friend` function is not a member function, it has no `this` pointer; it must qualify member access through an explicit object parameter (`cout << p.age;`).

Detailed Solutions — Programming Questions (Module 14)

CPP

```cpp
#include <iostream>
using namespace std;

class class_b;
class class_a {
    int a;
public:
    class_a(int v) : a(v) {}
    void show() const { cout << "a=" << a << " "; }
    friend void swap_private(class_a& x, class_b& y);
};

class class_b {
    int b;
public:
    class_b(int v) : b(v) {}
    void show() const { cout << "b=" << b << endl; }
    friend void swap_private(class_a& x, class_b& y);
};

void swap_private(class_a& x, class_b& y) {
    int temp = x.a;
    x.a = y.b;
    y.b = temp;
}

int main() {
    class_a obj1(10);
    class_b obj2(20);
    swap_private(obj1, obj2);
    obj1.show();
    obj2.show();
    return 0;
}
// Expected Output: a=20 b=10
```